```
  "email": "john@example.com",
  "phone": "555-555-5555"
}
```

Quite useful! We normally use Swagger for documenting the REST API. There is a lot of boilerplate in it. ChatGPT is perfect in such cases. I asked the following in the same conversation:

```
swagger doc for adding a new customer
```

Here is Swagger documentation for the POST */customers* endpoint that I described above:

```
/customers:
  post:
    summary: Add a new customer
    requestBody:
      content:
        application/json:
          schema:
            type: object
            required:
              - name
              - email
              - address
            properties:
              name:
                type: string
              email:
                type: string
                format: email
              address:
                type: string
    responses:
      201:
        description: The customer was successfully created
        content:
          application/json:
            schema:
              type: object
              properties:
                id:
                  type: integer
                  format: int64
                name:
                  type: string
                email:
                  type: string
                  format: email
                address:
                  type: string
      400:
        description: The request body is invalid
      409:
        description: A customer with the same email already
        exists
```

What else could I ask for? The final version of the documentation may not look this simple. Yet, it is a good starting point.

I was curious to see if ChatGPT could give me code for publishing the above REST API using Spring Boot. It did! The question was this:

```
spring controller for the above endpoint
```

And the response was:

```
@RestController
@RequestMapping("/customers")
public class CustomerController {

    @PostMapping
    public ResponseEntity<Customer> createCustomer(@Valid @
    RequestBody Customer customer) {
        // save the customer and return it
        Customer savedCustomer = customerRepository.
        save(customer);
        return ResponseEntity.status(HttpStatus.CREATED).
        body(savedCustomer);
    }
}
```

This is imperative Spring. Nowadays, we prefer the reactive paradigm. I asked the same thing with this question:

```
i want it with spring reactive
```

ChatGPT gave the starter code.

```
@RestController
@RequestMapping("/customers")
public class CustomerController {

    @PostMapping
    public Mono<ResponseEntity<Customer>> createCustomer(@
    Valid @RequestBody Mono<Customer> customer) {
        // save the customer and return it
        return customer.flatMap(customer -> customer
        Repository.save(customer))
           .map(savedCustomer -> ResponseEntity.status
           (HttpStatus.CREATED).body(savedCustomer));
    }
}
```

No modern development is completed without creating a Docker image. I took help from ChatGPT to generate a Dockerfile for the above code with this question:

```
dockerfile for the above application
```

And I got the following response, which is sufficient to start with:

```
FROM openjdk:11-jdk-slim
ARG JAR_FILE=target/*.jar
COPY ${JAR_FILE} app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Instead of asking for specific help, I just tried the fol-